



Prog C++

FS 2026 – Prof. Dr. Christian Werner
Autoren: Fabian Steiner
Mitwirkende: Mirko Ratti

1 Basics

1.1 Grundsätzlich

C++ ist sehr ähnlich zu C. Es gibt aber einige wichtige Änderungen zu C:

- zu benutzender Compiler heisst jetzt clang++
- Nativen Bool Typ : **bool**
- richtiges Konstanten Schlüsselwort : **constexpr**
- Standardbibliotheken haben kein .h mehr beim Aufrufen
- C Standartbibliotheken können weiter verwendet werden, haben aber ein C im Header(stdio.h → cstdio)
- Char-Literale (z. B. 'A') haben in C++ den Datentyp char (in C war es int)(Brauchen ein const)
- Es gibt benannte Namensräume. Wichtigster Namensraum für die STL: std

1.2 Hello world C++

```
1 #include <iostream> // iostream fuer Kommandozeilenausgabe
2 using namespace std; // dadurch entfaellt das "std::" vor cout
3 int main() // 'void' can be omitted
4 {
5     cout << "Hello World!" << endl; // "Einschieben" von hello
6     // world in den stream
7     return 0; // ein return 0 ist in c++ am Ende der main
8     // funktion fakultativ
9 }
```

1.3 Range-based for Loop

Range based for Loops laufen automatisch ein Array ab, ohne das zuerst die Grösse ermittelt werden muss. Die C for loop syntax wurde etwas erweitert. Syntax:

```
1 for(<DataType> <IteratorName> : arrayName){
2     ;//Loop where <IteratorName> is each element
3 }
```

Siehe: Als Array Name wird ein Pointer auf das Array verlangt, nicht auf das erste Element! Ein foreach funktioniert **nicht** mit einem **Pointerpointer**.

1.4 Streams

Ein Stream repräsentiert einen generischen sequentiellen Datenstrom. Z.b: ein Eingabefeld, Dateien oder Netzwerktraffic.

Die wichtigsten Operatoren sind:

- << : Inserter → Daten einfügen
- >> : Extractor → Daten herausholen

Für Standardklassen sind diese Operatoren bereits definiert.

Standardstreams

- **cin** : Standard Eingabe
- **cout** : Standard Ausgabe
- **cerr** : Standard Fehlerausgabe,
- **clog** : Standard Log Ausgabe, möglicherweise mit cerr gekoppelt

cin/cout Immer ganz links!, operatoren klar und lesbar anordnen

1.4.1 Streamformatierung

Formatierungen der Streams können mit folgenden Schlüsselwörtern erreicht werden:

Flag	Wirkung	Gültigkeit
boolalpha	bool-Werte werden textuell ausgegeben	Dauerhaft
dec	Ausgabe erfolgt dezimal	Dauerhaft
fixed	Gleitkommazahlen im Fixpunktformat	Dauerhaft
hex	Ausgabe erfolgt hexadezimal	Dauerhaft
internal	Ausgabe innerhalb Feld	Dauerhaft
left	linksbündig	Dauerhaft
oct	Ausgabe erfolgt oktal	Dauerhaft
right	rechtsbündig	Dauerhaft
scientific	Gleitkommazahl wissenschaftlich	Dauerhaft
showbase	Zahlenbasis wird gezeigt	Dauerhaft
showpoint	Dezimalpunkt wird immer ausgegeben	Dauerhaft
showpos	Vorzeichen bei positiven Zahlen anzeigen	Dauerhaft
skipws	Führende Whitespaces ignorieren	Dauerhaft
unitbuf	Leert Buffer dopo ogni operazione	Dauerhaft
uppercase	Kleinbuchstaben in Grossbuchstaben wandeln	Dauerhaft
<i>Benötigen Header <iomanip></i>		
setw(n)	Setzt minimale Feldbreite	Einmalig
setprecision(n)	Setzt Anzahl Stellen (signifikant o. Nachkomma)	Dauerhaft
setfill(c)	Setzt Füllzeichen (Standard: Leerzeichen)	Dauerhaft

1.5 Namespaces

- Namespaces helfen, Namenskonflikte zu verhindern. Sie fügen ein Namensvorsatz zu allen Variablen.
- Für jede Unit (Modul) sollte ein Namespace definiert werden
- **Mit kleine Buchstaben starten**
- Namenlose Namespaces: vermeiden static für lokale Variablen
- Der ::-Operator gibt immer die globalste Variable/Fkt. zurück

```
1 void f(); // name in global Namespace (1. f())
2 namespace myLib{void f(); int i} // namespace myLib (2. f())
3 f(); // f von global Namespace
4 myLib::f(); // f von namespace myLib
5 using namespace myLib;
6 using myLib::i; // posso usare using anche per singole var.
7 ::f(); // 1. f()
8 f(); // 2. f()
9
10 // Nameless namespace
11 namespace {
12     int localCounter = 0; // Invisibile all'esterno di questo
13     // file
14 }
15 void funzionePubblica() {
16     localCounter++; // instead of static
17 }
```

1.6 Kostanten

- #define soll nich mehr verwendet werden
- constexpr verwenden!
- enum nur benutzen, falls die +1-Automatik nützlich ist

1.6.1 Optiemierung constexpr

Bereich	Empfohlene Syntax
Header file	inline constexpr
Class or function	static constexpr

2 Funktionen

2.1 Default Argumente

- **Prototyp**: Standardwerte werden **nur** im Interface zugewiesen.
- **Weglassen**: Default-Parameter können beim Aufruf übersprungen werden.
- **Reihenfolge**: Default-Parameter müssen **immer am Ende (rechts)** der Parameterliste stehen.

```
1 void func(int a = 0, int b = 0, int c = 0){}
2 func(); // a = 0, b = 0, c = 0
3 func(1); // a = 1, b = 0, c = 0
4 func(1, 1); // a = 1, b = 1, c = 0
5 func(1, 1, 1); // a = 1, b = 1, c = 1
6 //Alle Funktionsaufrufe valide
7
8 void func2(int a, int b = 0, int c = 0); // korrekte
9 // Reihenfolge
```

Default Argumente können überall verwendet werden bis auf Aufrufparameter von Operatoroverloading. Dort sind sie **nicht** erlaubt. Dort machen sie aber ohnehin keinen Sinn.

2.2 Funzioni Inline

- **Makro-Ersatz**: Sichere Alternative zu C-Makros.
- **Zero Overhead**: Code wird direkt eingefügt (keine Funktionssprünge).
- **Type Safety**: Der Compiler garantiert die Typüberprüfung.
- **Ziel**: Kleine, sehr häufig aufgerufene Funktionen (z.B. Loops).
- **No-go**: Unanwendbar bei Rekursion oder Funktionspointern.
- **Header-only**: Wenn in Headern genutzt, Fkt.-Def. direkt ins file.h
- **Optimierung**: Erfordert aktive Flags (z.B. -O3).

```
1 // Definizione nell'Header (.h)
2 inline int quadrato(int x) { return x * x; }
3
4 // Compilazione con ottimizzazione
5 clang++ -O3 main.cpp
```

3 Objektorientierung

Objektorientierung soll es erleichtern, eine Abbildung der Realität zu erstellen. Viele Dinge aus der Wirklichkeit können als Modell dargestellt / simplifiziert werden. Diese Modelle können in Software in sogenannte Objekte umgewandelt werden.

3.1 Objekte

Objekte stellen Dinge, Sachverhalte oder Prozesse dar. Sie sind ein rein gedankliches Konzept.

Sie kennzeichnen sich durch:

- eine **Identität**, welche es erlaubt Objekte voneinander zu unterscheiden
- **statische Eigenschaften** zur Darstellung des **Zustandes** des Objekts in Form von **Attributen**
- **dynamische Eigenschaften** zur Darstellung des **Verhaltens** des Objekts in Form von **Methoden**

3.2 Klassen & Instanzen

Ähnliche Objekte können in **Klassen** zusammengefasst werden, um Programmieraufwand tiefer zu halten. Verwendete Objekte werden als **Instanz** eingesetzt. Diese Objekte haben dieselben Attribute, allerdings andere Werte.

Eine Beispielklasse:

```
1 #include <string> //Stl Strings
2 class Student {
3     public: // folgende Elemente sind Public
4         int IDNumber;
5         void doStuff(int time = 0);
6     private: // folgende Elemente sind Private(standart)
7         std::string name; //String
8         float bierkonsum;
9         unsigned long long int investedHoursInET;
10 };
11 // ----- start cpp -----
12 //implementierung der doStuff Funktion
13 void Student::doStuff(int time = 0) {
14 } //ToDo
15
16 int main(){
17     Student typETStudent; //instanz erzeugen
18     typETStudent.IDNumber = 1; //Attribute veraendern
19     typETStudent.doStuff(); //Methode rufen
20 }
```

3.2.1 Reihenfolge der Header-Datei (.h)

1. Dateikommentar mit Lizenzvereinbarung.
2. *Includes des verwendete System Header <...>*
3. *Includes der Projektbezogenen Header "..."*
4. *Definition von Konstanten*
5. *Typedef's und Definitionen von Strukturen*
6. *allenfalls extern-Deklaration von Global-Variablen*
7. *Funktionsprototypen, ink. Kommentar der Schnittstelle, bzw. Klassendeklarationen*

- Die *blauen* Punkte müssen sich im Includeguard befinden(#pragma once).
- Pro Header-Datei sollte nur eine Oberklasse deklariert sein.
- Kleine Unterklassen können im gleichen Header stehen.
- Kein **using namespace** ... in header files

3.2.2 Reihenfolge der Implementation (.cpp)

1. Dateikommentar mit Lizenzvereinbarung
2. includes der eigenen Header
3. includes der Projektbezogenen Header
4. includes der verwendeten System Header
5. allenfalls globale und statische Variablen
6. Präprozessor-Direktiven
7. Funktionsprototypen von lokalen, internen Funktionen (in nameless Namespace)
8. Definition von Funktionen und Klassen (**Kommentare nicht wiederholen**)

3.3 UML

Die Unified Modeling Language ist eine normierte Sprache um Objekte grafisch darzustellen, **programmierspracheunabhängig**.

3.3.1 UML-Klassendiagramm Notation

Besteht immer aus 3 Boxen:

Student
+ IDNumber : int # name : string - bierkonsum : float # investedTimeInET : int
- doStuff(time : int) + useless()

- + : public (überall sichtbar)
- - : private (nur in aktueller Klasse sichtbar)
- # : protected (in aktueller und Unterklassen sichtbar)
- *Italic* : Funktion oder Klasse ist Abstrakt (=0)
- Unterstrichen : Funktion oder Attribut ist static
- <Name der Klasse> : Konstruktor (ohne <>)
- ~<Name der Klasse> : Destruktor (ohne <>)

3.4 class vs struct

Eine class und struct können fast identisch verwendet werden. Eine class hat **standardmässig Sichtbarkeit private** sonst muss explizit mit **public**: gekennzeichnet, Struct public(wenn nichts definiert wird).

3.4.1 Zugriffsschutz

Modifikator	Beschreibung & Best Practice
public	Zugriff von innerhalb und von ausserhalb der Klasse. → Fast alle Methoden sind public. → Attribute sollten <i>nie</i> public sein.
protected	Zugriff von innerhalb der Klasse und den abgeleiteten Klassen . → Nur sehr sparsam zu verwenden.
private	Zugriff nur von innerhalb der eigenen Klasse aus. → Für alle Attribute und für spezifische lokale Methoden.

3.5 Inkludierte Dokumentation

Mann kann im H-File direkt die Dokumentation zu der Schnittstelle schreiben. Das ermöglicht das einfachere Verwenden der Schnittstelle.

```
1 /**
2  * @brief Kurzzusammenfassung der Klasse
3  *
4  */
5 class stuff{
6     public:
7         /**
8          * @brief Funktion von x
9          *
10         */
11         double x;
12         /**
13          *
14          * @brief Kurzzusammenfassung der Funktion
15          *
16          * @param eingang Funktion des Parameter
17          * @return nix, aber falls...
18          *
19          */
20         void doStuff(stuff eingang);
21 };
```

- Zu oberst der Name
- In der Mitte Attribute(Variablen)
- Unten Methoden.
- Methoden und Attribute fangen mit einem Symbol an, welches die **Sichtbarkeit** definiert und haben ein bestimmtes Merkmal:
- Wörter wie **void** o. **const** sind sprachspezifisch, nicht einfügen

Es kann immer mehr geschrieben werden, man sollte sich aber auf das Nötige beschränken.

4 Konstruktoren und Destruktoren

Konstruktoren (CTOR) und Destruktoren (DTOR) sind spezielle Methoden, die den Lebenszyklus des Objekts steuern. Beide haben den exakten Klassennamen und haben **keinen** Rückgabtyp (nicht einmal **void**).

4.1 Konstruktoren (CTOR)

Hauptaufgabe des CTORs ist die Baubereitstellung aller Attribute der Klasse.

- **Default-Konstruktor**: Hat keine Parameter. Wird automatisch aufgerufen, wenn das Objekt ohne Argumente deklariert wird (z.B. Stack s;). Falls nicht definiert, erstellt der Compiler einen implizit (der aber primitive Typen nicht initialisiert).
- **Copy-Konstruktor**: Erhält eine **const**-Referenz auf eigene Klasse (TString(const TString& s)). Wird benutzt, um ein Objekt als Kopie eines bestehenden zu erzeugen.
- **Explicit-Konstruktor**: Markiert man einen CTOR mit 1 Parameter als **explicit**, verhindert man ungewollte implizite Konversionen (es. TString s = 5;).

Werden verschiedene Konstruktoren definiert, muss der Default zuerst aufgerufen werden, wenn dieser erhalten bleiben soll.

4.1.1 Best Practices di Inizializzazione

Eine Hauptverantwortung des Konstruktors ist es, zu garantieren, dass das Objekt in einem validen Zustand entsteht. Das heisst, dass **alle** Klassen-Attribute auf einen definierten Wert initialisiert werden müssen.

- **Assegnamento nel corpo (4.4)**: Weniger effizient, Member werden erst mit Default-Werten erzeugt und danach überschrieben.
- **Initialisierungsliste (5.3)**: Die bevorzugte Methode aus Effizienzgründen; Attribute werden direkt bei der Erstellung initialisiert.
- **Member In-Class Initialization**: (Ab C++11) Erlaubt Default-Werte direkt in der Klassendeklaration zu setzen, was CTORs vereinfacht und Vergessen verhindert.

4.2 Gestione Memoria: Shallow vs. Deep Copy

Wenn eine Klasse Pointer auf dem Heap allokierten Speicher (**new**) verwaltet, ist das Default-Verhalten kritisch:

Shallow Copy (Default): Der Compiler kopiert lediglich die Adresse des Pointers. Beide Objekte zeigen auf denselben Speicher. Gibt einer ihn frei, zeigt der andere auf ungültigen Speicher (*Double Free* oder *Segmentation Fault*).

Deep Copy (Manuale): **Zwingend nötig** ist ein eigener Copy-Konstruktor, der neuen Platz auf dem Heap allokiert und die Nutzdaten kopiert.

4.3 Destruktor

Ein Destruktor entfernt eine Instanz aus dem Speicher, hat keine Aufrufparameter und auch kein Rückgabewert. Es kann immer nur **einen** Destruktor geben. Wenn der Destruktor fertig ist, sollte kein Speicher mehr vom Objekt belegt sein.

Destruktoren sollten immer **virtuell definiert werden** (siehe 10.2.1). Mit anderen virtuellen Funktionen **müssen** sie virtuel definiert werden. Der Funktionsname beginnt mit einer Tilde(~). Der Destruktor wird evtl. auch Implizit aufgerufen z.B., wenn aus einer Funktion wieder herausgesprungen wird.

4.4 Beispiel

```
1 // START H FILE
2 class Storage{
3     public:
4         Storage();
5         virtual ~Storage();
6         void add(int in);
7         void nix(Storage inC);
8     protected:
```

```
9     ...
10 private:
11     int* data;
12     int size;
13 };//eine Klasse die Int Werte speichert, aehnlich wie ein
    array
14
15 // END H FILE
16 // START Storage.cpp
17 Storage::Storage() { // Konstruktor
18     data = nullptr;
19     size = 0;
20 }
21
22 Storage::~Storage() { // Destruktor
23     delete[] data;
24     data = nullptr;
25     size = 0; // Speicher der allokiert wurde sollte hier
        freigegeben werden
26 }
27 void Storage::add(int in){
28     // todo
29 }
30 void Storage::nix(Storage inC){
31     // todo
32 }
33
34 // END Storage.cpp
35 // START main.cpp
36
37 int main(){
38
39     Storage* i1 = new Storage; // default konstruktor
40     Storage i2; // default konstruktor
41     i1->nix(i2); // Call-by-Value. Eine Kopie von i2 wird auf
        den Stack gelegt -> Copy-Konstruktor!
42
43     delete i1; // destruktur von i1 wird explizit aufgerufen
44     return; // destruktur von i2 wird implizit aufgerufen
45 }
```

5 Initialisierungslisten und direkte Initialisierung

Die Initialisierung von Datentypen kann auf verschiedene Arten erreicht werden. Es wird zwischen Zuweisung und Initialisierung unterschieden.

5.1 POD's

plain old datastructure / built in datatypes

```
1 // Zuweisung fuer POD:
2 int i; // Speicher fuer Instanz wird alloziert, Wert ist
    uninitialisiert
3 i = 5; // Wert (aus anderer Speicherstelle) wird zugewiesen
4
5 // Initialisierung fuer POD: Kein weiterer Speicherverbrauch
6 int i = 5; // (C-Style)
7 int i(5); // (C++-Style bis C++11)
8 int i{5}; // (neuer C++-Style)
9 // Aus Effizienzgruenden bevorzugen wir (fast) immer die (
    direkte) Initialisierung!
```

5.2 Non PODs

```
1 Aclass m;//kein Initialwert
2 // Vorsicht: Falls "Aclass" gross ist, muss viel kopiert
    werden
3 m = otherInstance;
4
5 // Copy-Initialisierung fuer non-POD:
6 Aclass m = otherInstance; //(copy ctor)
7 Aclass m(otherInstance); // C++-Schreibweise
8 Aclass m{otherInstance}; // C++-Schreibweise seit c++11
9
10 //Besser: direkte Initialisierung auch fuer non-PODs:
11 Aclass m("Eagle 1", 1, 2, ...); // (vor C++11)
12 Aclass m{"Eagle 1", 1, 2, ...}; // bevorzugte Schreibw. seit C
    ++11
13 // Die Instanz wird ohne Umwege mit den gewuenschten Werten in
    den Speicher geschrieben, sofern ein geeigneter user-
    defined ctor vorhanden ist!
```

5.3 User defined Constructor

Um eine Klasse richtig initialisieren zu können muss der Ctor korrekt definiert werden. Eine sogenannte **Initialisierungsliste** wird verwendet:

```
1 student::student(int _semester, long int _eltVerzweiflung):
2     semester{_semester},
3     eltVerzweiflung{_eltVerzweiflung}
4 //die Reihenfolge der Initialisierung ist durch die
    Reihenfolge im Speicher gegeben!
5 {
6     //Rumpf kann leer bleiben Initialisierung bereits fertig
7 }
```

6 Assertions

Assertions sollten Annahmen über korrekte Wertebelegung darstellen. Sie sind kein C spezifisches Konzept und sollten **nur zu Testzwecken** eingesetzt werden. Im Releasebuild sollten sie deaktiviert werden. Dafür gibt es spezifische Präprozessorflags. Der Header `<assert.h>` bzw. `<cassert>` stellt hierfür ein Präprozessor-Makro `assert` (Bedingung) zur Verfügung, um solche Zusicherungen auszudrücken.

Beispiel:

```
1 //define NDEBUG // Deaktiviert Assertions im Projekt
2 #include <cassert>
3 bool testStuff(int* in1){
4
5     assert(in1 != nullptr);
6     //assertion
7
8     ;//restliche implementation einer Funktion
9
10 }
```

Das obige Beispiel hat eine Assertion. Diese würde im Test, falls ein Nullpointer übergeben wird ein Fehler ausgegeben.

7 This-Pointer

Mit dem `this` Pointer kann ein Pointer des aufrufenden Objekt zurückgegeben werden. Bsp:

```
1 class Stuff{
```

```
2     public:
3         Stuff& funktion(int valIn);
4     private:
5         int value;
6 };
7 Stuff& Stuff::funktion(int valIn){
8     //do Stuff with val z.b. addieren auf interne Variabel
9     this->funktion(0); // auch moeglich
10    return *this; //this pointer -> wird dereferenziert und
        Referenz zurueckgegeben
11 }
12 int main(){
13     Stuff memb;
14     memb.funktion(1).funktion(2).funktion(3);
15 }
```

Nacheinander werden diese Funktionen durchlaufen da Funktion(2) vom zurückgegebenen Pointer aus Funktionsaufruf 1 aus gerufen wird. Die funktion wird **Kaskadiert**

8 Overloading

Operator Overloading bedeutet das bestehende Operatoren überladen werden im Sinne neu definiert / darüber geladen. Sie erhalten **neue Bedeutung** für eine Klasse.

8.1 Methodenoverloading

- **Definition:** Gleicher Name für Funktionen mit anderen Parametern (Typ, Anzahl, Reihenfolge).
- **Signatur:** Besteht aus *Name + Paramtern*. Der Rückgabety ist **exkludiert**.
- **Regel:** Unterscheiden sie sich nur im Rückgabety, generiert der Compiler einen Error.
- **Best Practice:** Overloading nur für vergleichbare Ops nutzen und **nie mit Default-Parametern vermischen**.

Eigenschaft	Overloading	Default Parameter
Speicher	Mehr Funktionen	Eine Funtkion
Wartung	Höher	Geringer
Flexibilität	Verschied. Typen	Vordefinierte Werte

```
1 // Approccio Overloading (3 implementazioni)
2 void print(int i);
3 void print(char c);
4 void print(int* arr, int n);
5 // A dipendena del parametro passato verra' chiamata una div.
    implementazione di print()
```

8.2 Operator overloading

Es können die inkludierten Operatoren von C++ überladen werden. Erlaubte Operatoren sind:

+	-	*	/	%	^	&		~
!	=	<	>	+=	-=	*=	/=	%=
^=	&=	=	<<	>>	<<=	>>=	==	!=
<=	>=	&&		++	--	,	->*	->
()	[]	new	delete	new[]	delete[]			

Nicht erlaubte Operatoren sind: `.'.*::?: sizeof typeid`. Es wird zwischen Overloading als *globalen Funktion* und als *Klassen-Methode* unterschieden.

8.2.1 Als Methode (Member function)

Ein Operator kann als Methode einer Klasse definiert werden. Dies ist der Standardfall, wenn der **linke Operand** ein Objekt der eigenen Klasse ist. Der linke Operand (Objekt selbst) ist dabei implizit, weshalb die Methode **einen Parameter weniger** annimmt als die globale Variante.

Syntax: <returntype> **operator**<typ>(type in);

- **Visibilità a livello di Classe:** In C++ agiert die Zugriffskontrolle (**private**) auf Stufe der **Klasse**, nicht des einzelnen Objekts.
- **Accesso Incrociato:** Eine Methode der Klasse A darf auf private Member jeglicher Instanz des Typs A zugreifen.
- **Applicazioni:** Diese Eigenschaft ist unverzichtbar um Operator Overloading (z.B. **operator+**), Copy-Konstruktoren oder Vergleichsmethoden zu bauen, wo ein Objekt interne Daten seines "Gleichen" lesen muss.

```
1 // --- H file ---
2 class Dozent {
3 private:
4     int number;
5
6 public:
7     Dozent(int n = 0) : number(n) {}
8
9     // 1. Metodo membro: Dozent + Dozent
10    Dozent operator+(const Dozent& other) const;
11
12    // 2. Metodo membro: Dozent + int
13    Dozent operator+(int val) const;
14 };
15
16 // --- CPP file ---
17 Dozent Dozent::operator+(const Dozent& other) const {
18     return Dozent(this->number + other.number);
19 }
20
21 Dozent Dozent::operator+(int val) const {
22     return Dozent(this->number + val);
23 }
```

Zwingend als Elementfunktion zu implementieren

- Zuweisung (=, +=, etc.)
- Index []
- Funktionsaufruf ()
- Zeigeroperator ->

8.2.2 Als globale Funktion (Global)

Globales Overloading wird **zwingend benötigt**, wenn der **linke Operand nicht von der eigenen Klasse ist**. Typische Beispiele sind:

- Ein primitiver Datentyp links (z.B. string + TString).
- I/O Stream-Operatoren wie <<, bei denen der linke Operand ein Stream-Objekt ist (z.B. std::cout << obj).

Syntax: <returntype> operator<type>(type val1, type val2);

Friend Keyword:

Oft müssen globale Operatorfunktionen auf private Daten der Klasse zugreifen. In diesem Fall werden sie innerhalb der Klasse mit dem Keyword **friend** deklariert. Dadurch erhalten sie als globale Funktionen Zugriff auf **alle privaten Attribute und Methoden** der Klasse, ohne selbst Teil der Klasse (Methode) zu sein. **Achtung: Friend-Funktionen sollten gezielt eingesetzt werden.**

```
1 // --- H file ---
2 class Dozent {
```

```
3     // La funzione amica permette l'accesso ai membri privati
4     // Funzione globale
5     friend Dozent operator+(const Dozent& d1, const Dozent& d2
6     );
7 private:
8     int numb;
9
10 public:
11     Dozent(int n) : numb(n) {}
12 };
13
14 // --- CPP file ---
15 Dozent operator+(const Dozent& d1, const Dozent& d2) {
16     return Dozent(d1.numb + d2.numb);
17 }
```

Beispiel I/O Streams (<<):

Damit Verkettungen wie cout << a << b; funktionieren, muss als Rückgabewert eine Referenz auf den ostream geliefert werden (return os;).

```
1 friend ostream& operator<<(ostream& os, const TString& s) {
2     os << s.getString(); // oder direkter Zugriff auf Attribute
3     return os;
4 }
```

8.2.3 Operatoren und Typumwandlungen

- Um unzählige Varianten eines Operators für jede Typkombination zu vermeiden, kann man **benutzerdefinierte Typumwandlungen** nutzen.
- Ein einzelner Operator kann diverse Inputs handhaben, falls ein Konstruktor existiert, der diese Typen in die Zielklasse wandelt.

```
1 // Dichiarazioni che possono essere sostituite
2 bool TString::operator<(const TString& s) const;
3 bool TString::operator<(const char* p) const;
4 friend bool operator<(const char* p, const TString& s);
5
6 // Possono essere ridotte a:
7
8 // Queste dichiarazioni sono sufficienti
9 TString(const char* p); // Typumwandlungs-Ctor
10 friend bool operator<(const TString& s1, const TString& s2);
```

9 Getter & Settermethoden

Getter und Settermethoden werden Typischerweise verwendet um Lese und Schreibzugriffe auf eine Klasse zu regeln. Attribute sind **alle Parameter einer Klasse**. Mit einer Get oder Set Methode kann der Zugriff auf diese kontrolliert / angepasst werden.

const after the funtion name declares that the method cannot modify any Attribute immer für Selector (Getter) benutzen

```
1 #include <string>
2 class Stuff{
3 public:
4     const std::string& getName() const;
5 protected://erlaubt schreibenden Zugriff von Unterklassen
6     void setName(const std::string& in);
7 private:
8     std::string name;//privates Attribut
9 };
10 const std::string& Stuff::getName() const{ // Getter
```

```
11     return name;
12 }
13 void Stuff::setName(const std::string& in){ // Setter
14     name = in;
15 }
```

9.0.1 Mutable

Gewisse Attribute dürfen von **const** Methoden geändert werden mit dem **mutable** Keyword, **so wenig wie möglich nutzen**

```
1 ...
2 mutable int error;
3 ...
```

9.0.2 Const correctness

- **Nur Lesen:** Ein als **const** deklariertes Objekt darf *nur* **const** Methoden aufrufen.
- **Sicherheit:** Ist eine Methode nicht als **const** markiert, nimmt der Compiler an, sie ändere das Objekt und verbietet sie auf konstanten Instanzen, um Seiteneffekte zu vermeiden.

Beispiel:

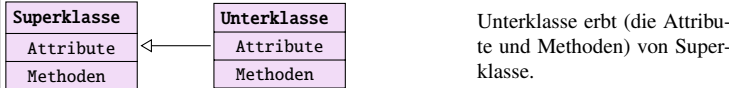
```
1 class Stack {
2 public:
3     int pop(); // Modifica lo stato (rimuove
4     elemento)
5     bool isEmpty() const; // Sola lettura (osserva lo stato
6     )
7 };
8 void check(const Stack& s) {
9     s.isEmpty(); // OK: il metodo promette di non modificare
10    s
11    // s.pop(); // ERRORE: pop() potrebbe modificare s
12 }
```

10 Vererbung

Vererbung erlaubt es Attribute und Methoden von anderen Klassen zu übernehmen. Die Oberklasse, Baisklasse oder Superklasse **vererbt** an eine Unterklasse, derived class oder eine Spezialisierung. Bei der Vererbung werden immer **alle** Attribute oder Methoden weitergegeben.

10.0.1 UML

In einem UML Diagramm wird vererbung wie folgt dargestellt:

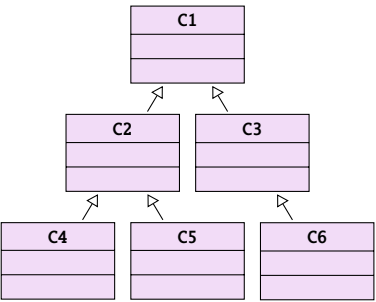


Dies stellt eine ist ein beziehung dar. Es ist sehr wichtig, dass die Pfeilspitze **genau so!** aussieht wie in der Darstellung da ansonsten Verwechslungsgefahr mit anderen Mechanismen entstehen könnte.

10.0.2 Syntax

```
1 class Unterklasse : public Superklasse{
2     ...
3 } ; // Unterklasse erbt Superklasse
```


10.0.3 UML++



Vererbung ist auch über mehrere Stufen Möglich:
Z.b. C4 Erbt alles von C2 und C1

10.0.4 Sichtbarkeit

Die Sichtbarkeit ist durch das Public definiert. Es ist möglich die Sichtbarkeit aller Attribute und Methoden einzuschränken. Mit **Public** wird alles mit der originalen Sichtbarkeit übernommen. Mit **protected** wird alles was public war protected. Mit **private** wird alles private (nicht sehr nützlich).

Vererbung	In Basisklasse	In abgeleiteter Klasse
public (normaler Fall)	public	public
	protected	protected
	private	private
protected	public	public
	protected	protected
	private	private
private	public	public
	protected	protected
	private	private

Wird eine Vererbung private durchgeführt, sind keine Attribute oder Methoden mehr sichtbar. Attribute soll nie **protected** sein

10.0.5 Constructor / Destructor chaining

Logica dei Costruttori (CTOR)

- **Prinzip der Verantwortung:** Der Konstruktor einer Klasse initialisiert explizit nur seine eigenen Attribute.
- **Delegation:** Für Basisklassen-Attribute wird der CTOR der Basisklasse selbst aufgerufen (die Basis "kümmert sich um ihre eigenen Dinge").
- **Prioritätsregel:** Die Elemente der Basisklasse müssen immer als erstes initialisiert werden.

```
1 Unterklasse::unterklasse(int _initvalOberklasse,
2                             int _initvallUnterklasse):
3     Oberklasse{_initvalOberklasse},// muss zuerst stehen da
4     unterklassenAttribut{_initvallUnterklasse}
5 {} // bleibt leer
```

Logica dei Distruttori (DTOR)

- **Umgekehrte Reihenfolge:** DTORs werden in der exakt umgekehrten Reihenfolge der CTORs ausgeführt.
- **Ausführungs-Reihenfolge:** Von unten nach oben (Subklasse → Basisklasse).

10.1 Überschreiben von Methoden

Geerbte Funktionen können überschrieben werden. Die Unterklasse definiert eine neue Methode mit demselben Namen. Allerdings kann es dann zu Mehrdeutigkeit kommen:

Im folgenden Beispiel haben eine Subklasse und eine Oberklasse eine print Funktion welche **nicht** virtual gekennzeichnet ist:

```
1 int main() {
2     Subclass p;           // Klassenerstellung
3     p.print();            // print von Subclass, ok
4     Subclass* sPtr = &p; // Subclass Pointer
5     sPtr->print();        // print von Subclass, ok
6     Topclass* tPtr = &p; // !! Topclass Pointer !!
7     tPtr->print();        // print von Topclass!!!!!!
8     Subclass& sRef = p;  // Subclass ref
9     sRef.print();        // print von Subclass, ok
10    Topclass& tRef = p;  // !! Topclass ref !!
11    tRef.print();        // print von Topclass!!!!!!
12 }
```

Dieses Beispiel soll zeigen, dass Pointer, welche eigentlich von einer Oberklasse auf eine Unterklasse zeigen auf die Eigenen Funktionen der Oberklasse zeigt, solange diese nicht überschrieben worden sind. Dies kann als Verhalten gewünscht sein.

10.2 virtual, final, override und scopeoperator

10.2.1 virtual

- **Zweck:** Kennzeichnet Methoden, die in Unterklassen überschrieben werden sollen. Dies ermöglicht **Dynamic Binding**, sodass zur Laufzeit immer die spezialisierte Methode der Unterklasse aufgerufen wird.
- **Polymorphie:** Eine Klasse, die mindestens eine virtuelle Funktion besitzt, wird als **polymorph** bezeichnet.
- **Vererbung:** Einmal als **virtual** deklariert, bleibt die Methode in der gesamten Vererbungskette virtuell.
- **Wichtige Syntax-Regel:** Das Schlüsselwort **virtual** darf nur bei der Deklaration (im H-File) verwendet werden.

10.2.2 Virtual beim Dtor

Ist eine Methode als virtual deklariert, so muss der Dtor **ZWINGEND** auch als virtual deklariert werden.

10.2.3 Override

Soll eine Methode eine geerbte Methode ersetzen so muss diese mit **Override** gekennzeichnet werden. Solch eine Methode ist automatisch auch virtual. Der Compiler kann so überprüfen, ob eine virtual Methode vorhanden ist.

10.2.4 Final

Final kann zum einen verbieten, dass eine Methode einer Klasse überschrieben wird (Logischerweise geht das nur bei virtuellen Methoden). Zum anderen kann sie auch das weitere Erben einer Klasse verbieten.

10.2.5 Scopeoperator

Per Scopeoperator kann (::) eine Methode aus einer Oberklasse gezielt aufgerufen werden. Mit diesem wird garantiert die Funktion welche definiert wird aufgerufen, egal ob diese aufgrund virtual überschrieben wird.

```
1 class Subexample : public Uppclass{
2     virtual ~Subexample(); // Correct Dtor
3     virtual void newMethod(); //gets overridden when inherited
4     // void oldVirtualMethod(); this inherited func doesnt get
5     // changed, it stays virtual
6     void getsOverridden() override;// overrides method in
7     // uppclass
8     virtual void iAmPerfect() final;// cant be overridden
9 };
10 void Subexample::newMethod(){
11     Uppclass::aMethodfromUppclass();
12     //calls a class from a class higher in the chain
13 }
```

10.3 Classi Polimorfe e Binding

- **Definition:** Eine Klasse ist polymorph, wenn sie mindestens eine **virtual** Fkt deklariert.
- **Datentypen:**
 - **Statischer Datentyp:** Der bei der Deklaration definierte Typ (Typ des Pointers oder der Referenz).
 - **Dynamischer Datentyp:** Der effektive Typ des Objekts zur Laufzeit (die echte Instanz).
- **Static Binding (Early Binding):** Passiert während der Kompilierung. Standard für nicht-virtuelle Fkt und hat keinen Overhead.
- **Dynamic Binding (Late Binding):** Funktionswahl passiert zur Laufzeit gemäss dynamischem Typ. Geht nur über Pointer o. Referenzen und hat leichten Overhead via V-Table.

10.3.1 Best Practice e Regole

- **Distruttori:** Müssen in Basisklassen zwingend **virtual** sein, um die korrekte Deallokation der Subelemente zu garantieren, speziell wenn über Pointer der Basisklasse verwaltet.
- **Costruttori:** Dürfen niemals als **virtual** deklariert werden.
- **Metodi:** Nur virtual deklarieren, falls Überschreiben geplant ist. Nicht-virtuelle Methoden dürfen in Subklassen nicht überschrieben werden.
- **Override:** In Subklassen müssen überschriebene Methoden explizit mit dem Keyword **override** markiert werden.

10.3.2 RTTI (Run-time Type Information)

RTTI erlaubt es, den Typ eines Objekts während der Ausführung zu identifizieren. Si utilizza la funzione **typeid()** inclusa nell'header <typeinfo>, welche ein Objekt vom Typ `std::type_info` zurückgibt.

```
1 #include <typeinfo>
2 int main() {
3     int a{0};
4     float b{0.0f};
5     if (typeid(a) != typeid(b)) { // is always true
6         // because they have a different type
7     }
8 }
```

10.3.3 Implementazione: La V-Table

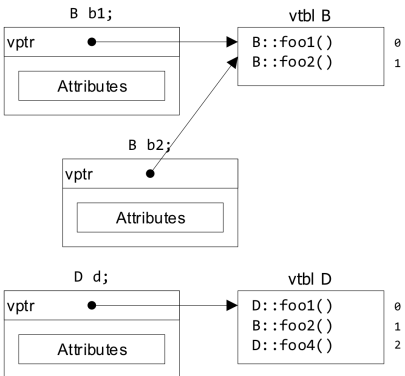
Die techn. Implementation von Polymorphismus und RTTI basiert auf der **V-Table**:

- Jede polymorphe Klasse besitzt eine V-Table mit den Pointern auf ihre virtuellen Funktionen.
- Jede Instanz der Klasse enthält einen versteckten Pointer (**V-pointer**), der auf die V-Table der eigenen Klasse zeigt.
- Zur Laufzeit nutzt die Instanz den V-pointer, um die Adresse der korrekten Funktion aufzulösen.

Enthält ein Basisklassen-Pointer-Array Objekte verschiedener Subklassen, so hat jedes Objekt einen V-pointer, der auf die spezifische V-Table der eigenen Klasse zeigt, was garantiert, dass

```
korrekte überschriebene Version-
1 class B {
2     public:
3         virtual void foo1();
4         virtual void foo2();
5         void foo3();
6     };
7
8 class D: public B {
9     public:
10        virtual foo1()
11        override;
12        virtual void foo4();
13    };
14
```

nen der Methoden aufgerufen werden.



10.4 Abstrakte Klassen

Wenn Klassen zwar festlegt, dass eine Methode da ist, diese aber noch nicht implementieren, nennt man das eine abstrakte Methode. Man spricht dann von einer abstrakten Klasse. Eine abstrakte Methode wird in UML *kursiv* dargestellt. Es ist egal, ob die abstrakten Methoden selbst definiert sind oder geerbt. Abstrakte Klassen kann man nicht instanziiieren. Es gibt kein spezielles Schlüsselwort dafür, man weist der Methode bei Definition 0 zu:

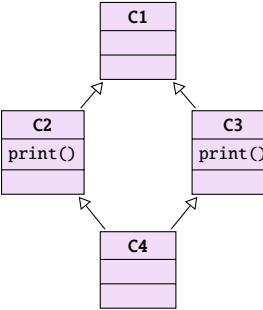
```
1 class ModernArt{// abstrakte klasse (wegen abstrakter Methode)
2     virtual void aMethod() = 0;// abstrakte Methode
3 };
```

10.5 Organisation der H files

Generell gilt immer noch: Jedes H File hat ein Cpp File. Allerdings können Unterklassen in ein H File zusammengefasst werden, falls diese nicht allzu Umfanglich sind. Ansonsten sollte ein neues H File erstellt werden. Für abstrakte Klassen fällt die Implementierung in einem Cpp file weg.

10.6 Mehrfachvererbung

Folgender Vererbungszweig ist möglich:



Solche Gebilde sind zwar möglich, sollten jedoch vermieden werden, da schnell sehr kompliziert und kaum lesbar. Es entsteht schnell Mehrdeutigkeit durch die verschiedenen Erbzweige.

Haben 2 Elternklassen eine Methode mit dem gleichen Namen, muss sie in der Subklasse neu definiert werden, um das richtige Verhalten festzulegen (ad es. C1::print())

Bei der Definition einer Klasse können weitere Oberklassen mit einem Komma getrennt angegeben werden

```
1 class Unterklasse : public Oberklasse1, public Oberklasse2{};
```

10.6.1 Diamond Problem

- **Das Problem:** Ohne Vorkehrung hätte C4 zwei getrennte Kopien von C1 (via C2 und via C3). **Erzeugt Mehrdeutigkeit beim Member-Zugriff.**
- **Die Lösung (Virtual Inheritance):** das Keyword **virtual** in den mittleren Klassen (C2 und C3) weist den Compiler an, eine geteilte Instanz von C1 zu bauen.
- **Reihenfolge der Initialisierung:** Folgt einer hierarchischen Priorität:
 1. **Virtuelle Basis:** C1 wird als erstes initialisiert.
 2. **Nicht-Virtuelle Basen:** C2 und dann C3 (gemäß der Deklarations-Reihenfolge in C4).
 3. **Abgeleitete Klasse:** Der Body vom Konstruktor von C4.

```
1 class C1 { ... };
2
3 // Il virtual va inserito qui (classi intermedie)
4 class C2 : virtual public C1 { ... };
5 class C3 : virtual public C1 { ... };
6
7 // C4 eredita normalmente; ordine ctor: C1 -> C2 -> C3 -> C4
8 class C4 : public C2, public C3 {
9     public:
10         C4() : C1(), C2(), C3() { ... }
11 };
```

10.7 Static

Static im Zusammenhang mit Klassen bindet eine Methode oder ein Attribut an die Objektklasse und nicht an die Objektinstanz (ähnlich zu Python Klassenvariablen).

10.7.1 Methoden

Static Methoden müssen im Header-File deklariert und im Cpp-File definiert werden. Sie dürfen nur auf Static definierte Attribute zugreifen da **keine Objektinstanz vorhanden**. Hauptanwendungen sind Hilfsfunktionen wie Funktionsaufrufcounter oder Umrechnungsfunktionen. Bei der Deklaration wird der Klassennamen verwendet (siehe Bsp).

10.7.2 Attribute

Static Attribute müssen im H deklariert **und im Cpp File** definiert werden. Sie werden verwendet für z.B. Objektspezifische Konstanten oder im Zusammenhang mit Static Counter. (*class scope*)

10.7.3 Bsp

```
1 // Logger.h
2 class Logger {
3     public:
4         static void log(const std::string& message);
5     private:
6         static int nrObjects; // Non inizializzabile dirett.
7 };
8 // Logger.cpp
9 int Logger::nrObjects = 34; // Necesario!
10 // main.cpp
11 int main() {
12     // Benutze log aus Logger, ohne Instanz:
13     Logger::log("C++ ist super!"); // class name wird verwendet
14     !
15     return 0;
16 }
```

11 Memorymanagement

Es gibt verschiedene Gründe warum nicht immer gleich viel Speicher verwendet wird z.B. da nicht unbegrenzt vorhanden ist. Speicher wird nur dann verwendet, wenn er wirklich gebraucht wird. Das Memorymanagement muss aktiv beachtet werden. Es wird zwischen automatischen und manuellen Speichermanagement unterschieden.

11.1 Automatisch

Automatisches Speichermanagement wird anhand der Lebenszeit einer Variable festgestellt und während dem Programmieren bereits festgelegt. Der Speicher liegt auf dem sogenannten **Stapel / Stack**.

Wird eine Funktion aufgerufen, werden die benötigten Variablen auf dem Stack definiert. Ebenso wird eine Returnadresse zur Funktion, die die Funktion aufgerufen hat abgelegt. Wie die Daten angeordnet werden, ist Architektur-abhängig. Wird die Funktion wieder verlassen, werden die Variablen vom Stack wieder entfernt und zur aufrufenden Funktion zurückgesprungen.

11.2 Manuell/dynamisch

Man kann auch manuell Speicher anfordern. Zum Beispiel, falls zur Compilezeit nicht bekannt ist, wie viel Speicher gebraucht wird. Dieser Speicher ist auf dem sogenannten **Haufen / Heap**.

Der Heap ist ein Speicherbereich, der vom Betriebssystem bereitgestellt und verwaltet wird.

Zur Laufzeit wird Speicher dynamisch angefordert. Dieser bleibt so lange belegt bis dieser Manuell wieder freigegeben wird.

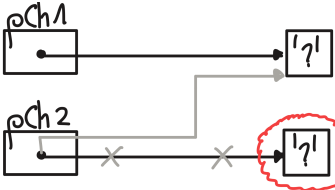
Es kann aber auch sein, falls der Speicher stark fragmentiert ist, das kein Speicher bereitgestellt werden kann. Generell ist bei sehr vollem Speicher das Arbeiten erschwert.

11.2.1 Memory leaks

Falls ein Programm unkontrolliert Speicher aufnimmt oder den ihm zur Verfügung gestellte nicht wieder freigibt, spricht man von einem Speicherleck. Diese sind zu verhindern da diese zu einer Systemüberlastung und Abstürzen führen.

Beispiel:

```
1 char* pCh1 = new char;
2 char* pCh2 = new char;
3
4 std::cin >> *pCh1;
5
6 pCh2 = pCh1;
7 // pCh2 points now to the Ch1
  memory cell
8 // The access to the memory
  cell of pCh2
9 // is now lost (memory leak!)
10
11 delete pCh1;
12 delete pCh2;
13 // ERROR, already with pCh1
  freed
```



11.3 Speichermanagement funktionen

Es gibt einige Funktionen für Speichermanagement. In C und C++ sind diese leicht verschieden. Generell geben diese einen Pointer zurück, welcher auf freien Speicher zeigt. Diese müssen immer auf einen Nullpointer geprüft werden, das keine Garantie vorhanden ist, ob überhaupt Speicher vorhanden ist! Generell sollte mit solchen Pointer immer misstrauisch gearbeitet werden und nach Abschluss immer wieder freigegeben werden.

11.3.1 C: malloc(), calloc(), free()

malloc() gibt einen Voidpointer(pointercasting nicht vergessen) zurück welche auf eine angeforderte Grösse an Bytes Speicher zeigt. Der Speicher ist nicht initialisiert calloc() macht dasselbe, initialisiert aber den Speicher auf 0. free() gibt den Speicherbereich wieder frei.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void *malloc(size_t size);
5 // Alokiert Speicherbereich size in Bytes
6 void *calloc(size_t size);
7 // Alokiert Speicherbereich size in Bytes und setzt diesen 0
8 free(void *ptr);
9 // Gibt Speicherbereich auf welcher ptr zeigt wieder frei
10
11 int main(){
12     int* iPtr = NULL;
13     iPtr = (int*)malloc(3*sizeof(int));
14     // Speicher fuer 3 Ints reservieren
15     // Insert Nullpointer check here!!!
16     free(iPtr);
17 } // Speicher wider freigegen
```

Falls kein Speicher allokiert werden kann, wird ein NULL-pointer zurückgegeben.

11.3.2 C++: new, delete, new[], delete[] (Operatoren)

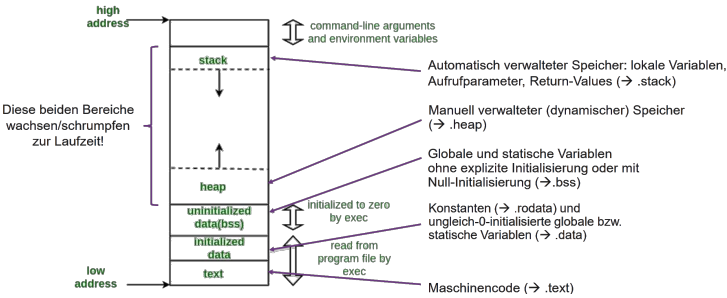
Der new-Operator erstellt ein Pointer welcher auf die mitgegebene Grösse zeigt. new[] macht dasselbe, ausser das dieser ein Array zurückgibt. delete / delete[] gibt den Speicher wieder frei. Dieser kann auch auf den Nullpointer ausgeführt werden. Dieser ist Delete egal.

```
1 int main(){
2     int* intPtr = new int;
```

```
3 // Allokiert fuer intPtr ein Speicherbereich eines Ints
4 int* intArrayPtr = new int[10];
5 // Allokiert fuer intArrayPtr ein Speicherbereich eines
  Int Array der groesse 10
6 if(intPtr == nullptr) return -1; // Nullpointercheck
7
8 delete intPtr; // Gibt intPtr wieder frei
9 delete[] intArrayPtr; // Gibt intArrayPtr wieder frei
10 intArrayPtr = intPtr = nullptr
11 }
```

11.4 Speicheraufbau

Der Speicheraufbau bildlich dargestellt sieht ungefähr so aus:



Je nach Architektur kann es auch sein, dass die hohen und tiefen Adressen anders herum sind.

11.5 Referenzen

Eine Referenz ist ein Alias (ein alternativer Name) für eine bereits bestehende Variable oder Speicherzelle. Sie stellen eine modernere, sicherere und striktere Version von Pointern dar.

11.5.1 Caratteristiche Principali

- **Zwingende Initialisierung:** Muss per sofort initialisiert werden int& r = x;
- **Unveränderlichkeit:** Darf nach Definition nicht modifiziert werden, um auf ein anderes Objekt zu zeigen.
- **Sicherheit:** Dürfen niemals uninitialized sein und besitzen niemals den Wert nullptr.
- **Effizienz:** Können in CPU-Registern liegen und belegen nicht in jedem Fall physischen Speicher. Der sizeof()-Operator gibt die Grösse des Typs zurück, auf welchen die Referenz zeigt.
- **Syntax:** Werden wie normale Variablen benutzt, ohne eine Notwendigkeit zur De-referenzierung.
- **Limitierung bei Arrays:** Referenzen können nicht genutzt werden, um Arrays via Arithmetik (Inkrement/Dekrement) zu durchlaufen. Für genau diesen Zweck bleiben die Pointer unverzichtbar.

11.5.2 Confronto: Value vs Reference

Caratteristica	Call by Value	Call by Reference
Datenkopie	Ja (sehr teuer bei grossen Objekten)	Nein (übergibt Alias)
Ändert Original	No	Ja
Sintassi	void f(int a)	void f(int& a)

11.5.3 Best Practices

Die Nutzung von Referenzen senkt die Fehlerrisiken im Vergleich zu Pointern drastisch. Dennoch bleiben Pointer in ganz bestimmten Fällen unumgänglich:

- **Array-Verwaltung:** Ein Array berfälltin Pointer auf das erste Element (pointer decay), was Pointer zum Standard-Tool für das Durchlaufen o. Bearbeiten am Stück liegender Speicherblöcke macht.
- **Low-Level Programmierung:** Nötig für die direkte Hardware-Interaktion oder für die manuelle dynamische Allokation (new/delete).

```
1 // 1. Usa const per oggetti grandi in sola lettura
2 // Ottimizza le prestazioni impedendo modifiche accidentali
3 int foo(const BigType& b);
4
5 // 2. Passa SEMPRE struct e classi per referenza
6 void process(MyClass& obj);
7
8 // 3. ERRORE DA EVITARE: mai tornare referenze locali
9 int& func() {
10     int x = 10;
11     return x; // PERICOLO: x viene distrutta all'uscita!
12 } // La referenza diventerebbe "dangling" (punta al vuoto)
```

11.6 Speicherplatzbedarf von Objekten

Eine Unterklasse braucht immer mindestens so viel speicher wie eine Oberklasse.

11.6.1 Zuweisungen

Zuweisungen von Oberklasse zu Unterklasse sind in der regel möglich da alle geerbten Attribute ja vorhanden sind. Umgekehrt geht das allerdings nicht, da der Speicher dafür nicht initialisiert ist.

11.6.2 Slicing Problem

- **Definition:** Tritt auf, falls ein Subklassen-Objekt als Value einer Funktion übergeben wird, welche aber explizit eine Superklasse erwartet.
 - Dabei wird jener Copy-Constructor der Superklasse aufgerufen.
 - Nur der "BasisTeil vom Objekt wird kopiert; die Erweiterungen jener Subklasse gehen verloren (Slicing).
- **Conseguenza:** Innerhalb der Funktion agiert das Objekt nun exklusiv als Basisklassen-Instanz und verliert jegliches polymorphes Verhalten absolut.
- **Soluzione:** Nutze für die Übergabe immer eine Referenz (const T&) oder mache es via Pointer. Das erhält die Integrität vom Original-Objekt und erlaubt das korrekte Funktionieren der virtual-Methoden.

11.7 Type Casting

Cast:	Anwendungsbereiche	Wissen wertet	Beispiele
implicit: kein Cast	- Numerische Typen in bool (z.B. 0 ist falsch, 1 ist wahr) - Objektinstanzen einer Unterklasse in einer Variablen vom Typ der Oberklasse (z.B. <code>Animal* a = new Cat;</code>) - Pointertypen, deren Umkonvertierung offensichtlich ist - die genau umgewandelt werden - Alle, was implizit möglich ist, unternimmt dabei gewisse Warnungen		1. signed char a = 1; int b = a; 2. int a = 1; char b = a; // ggf. mit Warnung 3. float b; Animal a = b; // Upcast: a ist ein Animal
static	- Unterstützt die Konvertierung von Instanzen von Ober- und Unterlassen in beide Richtungen - unumkehrbar: Upcast und Downcast - Laufzeit, Auswertung ausschliesslich zur Compilezeit	Um einen grösseren Integer in einen kleineren zu konvertieren, ist bei der Compiler überschlüssige Bits einfach ab und behält nur die am wenigsten signifikanten (LSB).	1. int a = 1; char b = a; // cast: char >= 0; 2. float b = new Bird; Animal* a = b; // geht implizit! Bsp: c = static_cast<Bird*>(a); // c ist vom Typ Bird, aber a ist vom Typ Animal // weil nicht jedes Animal ein Bird ist! //
dynamic	- Pointer- und Referenztypen auf polymorphen (Obe- und Unterlassen in beide Richtungen) verwenden - Laufzeit, Auswertung zur Laufzeit	Unterstützt umkehrbar den Upcast für Pointer und Referenztypen auf nichtpolymorphe Klassen (immer dann möglich, wenn wir dies abcasten) Fehl der dynamic_cast (fehlschlüssig, ist fertig) eine statische cast (kompilieren) geworden	Animal* a = new Ant; Bird* b = dynamic_cast<Bird*>(a); // b ist vom Typ Bird, aber a ist vom Typ Animal // immer zur Laufzeit ausgewertet, dh: b == nullptr //
const	- Verändert die Konstante und nicht die Variable - Ziel eines Pointer oder Referenz Typs, sowie zwischen beteiligten Pointer und int-Typen		const int val = 10; // val ist vom Typ const int, aber nicht int b1 = const_cast<int*>(&val); Animal* a = new Ant;
reinterpret	Kann auch mit Referenz-Typen umgehen	Achtung: erzeugt plattformspezifisches Verhalten	uint8_t* b = reinterpret_cast<uint8_t*>(a);

Die C Syntax für Casting sollte nicht mehr verwendet werden da meistens nicht optimales Verhalten.

11.8 Templates

- Typ-Unabhängigkeit:** Sie erlauben generischen Code mittels **Placeholdern** (Platzhaltern) anstelle der Nutzung von spezifischen Typen.
- Typen-Sicherheit (Type Safety):** Im Ggs. zu C (**void***) garantieren Templates eine Typenprüfung durch **Early Binding** bei der Kompilation.
- Instanziierungs-Parameter:** Ausser Typen können auch **Konstanten** direkt als die expliziten Template-Parameter übergeben werden.
- Vielseitigkeit:** Dürfen auf **Klassen** wie auch **Fkt.** angewendet werden.
- Auswertung zur Compilezeit:** Der Compiler generiert den eigentlichen Code direkt während der **Compile-Time**-Phase.
- Struktur und File:** Erfordern Deklaration, Definition und Parameter; meist **vollständig in den Header-Files (.h / .hpp)** definiert.

- Code-Erzeugung:** Der Compiler baut eine separate Kopie der Funktion oder Klasse für absolut jeden **unterschiedlichen Typ**, der genutzt wird.
- Zero Dead Code:** Wird ein Template nie im Code genutzt, generiert es exakt keine einzige Anweisung im allerletzten finalen Binary.
- Memory-Footprint und Code Bloat:** Eine Instanziierung für sehr viele Typen oder die übermässige Nutzung vom Keyword **inline** für lange Defs. verursacht ev. **Code Bloat**.

11.8.1 Implementation

Die Typendefinitionen sind dann durch das T zu ersetzen.

```
1 // H-File
2 template<typename T>
3 void swap(T& a, T& b); // deklaration
4
5 template<typename T> // definition (immer .h)
6 void swap(T& a, T& b) { // REFERENCES for parameter
7     T temp{b};
8     b = a;
9     a = temp;
10 }
```

As the dimension of the paramters is unknow ALWAYS use references

11.8.2 Syntax

```
1 // Template-Definition fuer Funktionsdeklaration:
2 template<typename type1, typename type2, ...>
3 returntype name(type1 param1, type2 param2 , ...);
4 // Inline goes before returntype
5
6 // Template-Definition fuer Funktionsdefinition:
7 template<typename type1, typename type2, ...>
8 returntype name(type1 param1, type2 param2 , ...) { ... }
9
10 // Klassentemplate:
11 // Template-Definition fuer Klassendeklaration:
12 template<typename type1, typename type2, ...>
13 class Classname{ ... };
14
15 // Template-Definition fuer Methode einer Klasse:
16 template<typename T1, typename T2, ...>
17 returntype Class-name<T1, T2, ...>::method-name(type1 param1,
18     type2 param2 , ... ) { ... };
```

11.8.3 Instanziierung

Instanziierung ist der Prozess, mit dem der Compiler hier dann den konkreten Maschinencode generiert

- Implizit:** Der Compiler leitet den Typ aus übergebenen Parametern ab. **Wichtig:** Der Return-Typ wird nie für die Ableitung berücksichtigt.
- Explizit:** Der Typ wird manuell innerhalb der spitzen Klammern definiert.

Klassentemplates (<> obligatori):

Typ muss spezifiziert sein, damit der Compiler den effektiven Speicherbedarf ermitteln kann.

Stack<int, 10> s1; // Instanziert einen Stack von 10 Integern

Funktionstemplates (<> optional se deducibili):

Ist der Typ via Parameter glasklar, können diese spitzen Klammern weggelassen werden.

```
int i = minimum(iF, size); // Implizite Typ-Deduktion fÄ¼r int
int i = minimum<int>(iF, size); // Explizite Qualifikation
```

11.8.4 Overloading

Die Funktions-Templates können auch mit anderen Templates oder "normalen" Funktionen überladen werden. Bei gleichnamigen Aufrufen wertet der Compiler kompatible Fkt. aus, instanziiert jene der Templates, fügt sie den verfügbaren normalen Fkt. bei und wählt jene Variante, die den besten **Match** für Parameter-Typen bietet.

11.8.5 Kompilierung und Code-Organisation

- Header-Only:** Compiler muss Def. bei jeder Übersetzung Behen". Hauptnachteil ist eine **erhebliche Zunahme jener finalen Kompilierzeiten**.
- Explicit Instantiation (.cpp):** Um die Kompilierzeiten zu verringern, kann man Definitionen in ein .cpp-File auslagern und Instanziierung forcieren (es. **template class Stack<int>;**).

12 Exceptions (Ausnahmen)

- Error (Fehler):** Ein Implementierungsfehler. Er sollte zwingend während dem Testing/Verify behoben werden.
- Exception (Ausnahme):** Eine abnormale Bedingung, welche in Runtime auftreten kann (Bsp. File fehlt, Network-Timeout).

12.1 Meccanismo: Throw e Catch

Eine Exception ist ein Objekt, per throw Keyword geworfen und per catch gefangen.

- Throw by value:** Diese Exceptions sollten stets zwingend per Wert geworfen werden.
- Catch by const reference:** Um unnötige Kopien und das Objekt-Slicing-Problem zu vermeiden, solltest du sie immer als konstante Referenz fangen.
- Stack Unwinding:** Wurde Exception geworfen, zerstört das Programm alle Objekte (ruft Destruktoren auf) und springt direkt **zum ersten kompatiblen Handler**.

```
1 try {
2     if (errorCondition) {
3         throw MyExceptionClass{"Error string"}
4     }
5 } catch (const MyExceptionClass& exc) { // Always const ref.
6     // Error handling.
7 }
8 // normal execution continues here
```

12.2 Propagazione (Exception Propagation)

Wird eine Exception lokal nicht gehandelt, reicht man sie "nach oben" weiter (z.B. auf die Applikations-Ebene), wo ein sinnvoller Entscheid fällt (Grundsatz: Handle nur das, worüber du entscheiden kannst).

- Innerhalb von einem catch-Block wirft die throw;-Instruktion (ganz ohne Argumente) die genaue, aktuelle Exception erneut an die aufrufende Funktion.

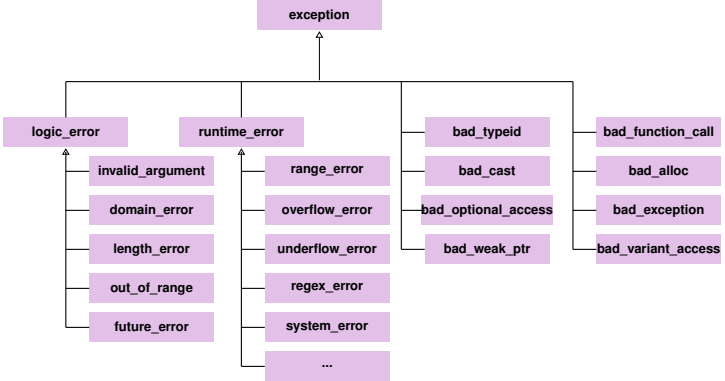
12.3 Gerarchia e Ordine dei Catch

In C++ werden Exceptions hierarchisch unter der Basisklasse `std::exception` (ist in `<exception>` definiert) angeordnet.

- logic_error: NICHT BENUTZEN** Fehler, die in der Entwicklung vermeidbar sind
- runtime_error:** Unvorhersehbare Fehler in Folge von externen Ereignissen (es. `overflow_error`, `system_error`, **include `stdexcept`**).

Cruciale Regel: Handler müssen zwingend von sehr spezifisch zu sehr generisch im Code geordnet sein. `std::exception` oder der Catch-All ... ist am Schluss.

```
1 catch(...) { /* Catch everything */ }
```

Beispiel:

```
1 #include <stdexcept> // For most runtime errors
2
3 int main() {
4     int x;
5     std::cin >> x;
6     try {
7         a(x); // 'a' can throw various types of exceptions
8     } catch (const char* e) {
9         std::clog << e << std::endl; // handling Type const char*
10    } catch (const std::range_error& e) {
11        std::cerr << i << std::endl; // handling range_error
12    } catch (const std::runtime_error& e) {
13        std::cerr << i << std::endl; // handling runtime_error
14    } catch (...) {
15        // handling everything
16        std::cerr << "other exception occured" << std::endl;
17    }
18    return 0;}
```

12.4 Eccezioni non gestite

Wird absolut gar kein gültiger Handler im *Call Stack* gefunden:

1. Wird nun die Funktion namens `std::terminate` gerufen.
2. `std::terminate` ruft einen `terminate_` handler (Default ist `std::abort`).
3. Das Programm endet jetzt ziemlich brüsk (kein Ausführen der Destruktoren)

Es ist möglich, dieses Verhalten spezifisch über `std::set_terminate()` anzupassen.

12.5 noexcept

- **noexcept:** Eingeführt in C++11, zeigt es, dass Fkt garantiert keine Exceptions wirft. Dies ist sehr nützlich für die Optimierung des Compilers.
- **Operatore noexcept(expr):** Gibt exakt `true` zurück, wenn die Expression als `noexcept` deklariert ist.

```
1 void foo() noexcept; // Es kan KEINE Exception auftreten
2 void foo2(); // es KONNEN exceptions auftreten
3 void bar() noexcept(false); // es KONNEN exceptions auftreten
4
5 bool a = noexcept(foo); // TRUE
6 bool b = noexcept(foo2); // FALSE
```

12.6 C++ e Low-Level Exceptions

Anders als in Java oder C# übernimmt die Sprache C++ **kein** auto *Mapping* jener Low-Level Exceptions (wie z.B. bei der Division durch Null) in normale Sprach-

Exceptions.

Handling con `catch(...)` { }

13 STD Library

13.1 array

- **Definition:** `std::array<T, size>` ist ein Container von absolut fester Grösse implementiert als Template-Klasse.
- **Dimension:** `std::array` weiss seine ganz exakte Dimension aus der Memberfunktion `.size()`.
- **Zugriff auf Daten:** Klassische eckige Klammer Notation in `[]`.
- **Sicherheit:** `.at()` greift dank Limit-Check sicher zu, Exeption `std::out_of_range` tritt bei einem ungültigen Zugriff auf.
- In den Parametern von Fkt übergebbar als eine **Referenz, ist stets zu tun** (da es ja Klassen sind)

Beispiel:

```
1 #include <array> // Mandatory to use std::array!
2 template <typename T, std::size_t n>
3 void print(const std::array<T, n>& arr) {
4     for (size_t i = 0; i < arr.size(); ++i) {
5         std::cout << arr[i] << " ";
6     }
7     std::cout << std::endl;
8 }
9
10 int main() {
11     std::array<int, 5> number = {1, 2, 3, 4, 5};
12     std::array<double, 3> dnumber = {1.45, -2.32, 345.432};
13
14     std::cout << "Third elem.: " << number.at(2) << std::endl;
15 }
```

13.1.1 Matrix

- **Struktur:** Externe Dimension: Zeilen, intern: die Spalten.
- **Erfordert zwei Doppel-Paare an geschweiften Klammern zum Initialisieren** (weil dieses `std::array` eine Klasse ist)
- **Zugriff:** Doppelte Index Notation: `matrix[riga][colonna]`.

Beispiel:

```
1 template <typename T, std::size_t m, std::size_t n>
2 void printMatrix(const std::array<std::array<T, m>, n>& matrix
3 ) {
4     for (std::size_t i = 0; i < matrix.size(); ++i) {
5         for (std::size_t j = 0; j < matrix[i].size(); ++j) {
6             std::cout << matrix[i][j] << " ";
7         }
8         std::cout << std::endl;
9     }
10 }
11
12 int main() {
13     // Definizione di una matrice 2x3 (2 righe, 3 colonne)
14     std::array<std::array<int, 3>, 2> matrix = {
15         {{1, 2, 3}}, // Riga 0
16         {{4, 5, 6}}}; // Riga 1, ATTENZIONE Doppia }
```

13.2 vector

`std::vector` ist ein Klassentemplate welches die Kapazität selbständig bei Bedarf erweitert.

```
1 #include <vector>
2 std::vector<dt> v; // Template instanzieren mit <datentyp> (dt)
3 v.push_back("Value"); // Value anfüegen
4 v.erase("iterator"); // Elemente loeschen (Iterator=fancy Pointer)
5 v.begin(); // returned ein Pointer auf das erste element
6 v.end(); // returnd ein Pointer HINTER das letzte Element
7
8 for (const dt s : v) { // Iteriert ueber Elemente vom Vector
9     doSomething(s); } // do something mit element
10
11 for (std::vector<dt>::iterator it=v.begin(); it!=v.end(); it++)
12     { doSomething(s); } // do something mit element, altes for
```

14 Makefiles

Makefiles sollten generell das Umsetzen des Codes in Maschinencode vereinfachen. Es ist eine Skriptingsprache welche grob nach dem Muster *Erzeugnis/target* : Abhängigkeiten/Dependency folgt. Dann folgt indentiert der Befehl, um dieses Erzeugnis zu erzeugen.

Wenn ein Erzeugnis keine Datei zurückgibt, muss dieser als `.PHONY` markiert werden. Das verhindert, dass eine Datei mit demselben Namen das Ausführen verhindert.

Am besten sieht man das an einem Beispiel:

```
1 all : stuff # erzeugt die Definition fuer make
2     all
3
4 project : main.o lib.o # Projekt linken
5     clang++ -Wall -o vector Vector3D.o main.o
6
7 lib.o : lib.cpp # lib.cpp compilen
8     clang++ -Wall -c lib.cpp
9
10 main.o : main.cpp # main.cpp compilen
11     clang++ -Wall -c main.cpp
12
13 clean : # Projekt cleanup
14     rm -f project.exe lib.o main.o
15
16 .PHONY : clean all # Mariert die Targets "clean" und
17     "all" als "PHONY" -> Lesbarkeit
```

Der Befehl `make all` baut nun das Projekt.

Sind die `o` Dateien noch aktuell werden diese nicht erneut kompiliert. Das Projektverzeichnis kann einfach durch `make clean` aufgeräumt werden.

14.1 Platzhalter

In einem Makefile können auch Platzhalter verwendet werden:

- `$@` Dateiname des Targets
- `$<` Dateiname der ersten Dependency des aktuellen Targets
- `$^` Dateiname aller Dependencies des aktuellen Targets durch Leerzeichen getrennt

Mit diesen Mitteln kann ein Professionelleres Makefile erstellt werden, welches relativ universell eingesetzt werden kann:

```
1 CXX = clang++ # Verwendeter Compiler
```

```
2 CXXFLAGS = -Wall -c      # compilerflags
3 LDFLAGS = -o             # loader flags
4 BIN = main               # binary output / Name des .exe
                           file
5 OBJS = drink.o drinktest.o # objectfiles
6
7 all: $(BIN)              # Make all befehl
8
9 %.o: %.cpp               # cpp Files zu o dateien compilen
10  $(CXX) $(CXXFLAGS) $<
11
12 $(BIN): $(OBJS)          # Linken zu exe file
13  $(CXX) $(LDFLAGS) $@ $^
14
15 run: $(BIN)              # Projekt ausfuehren
16  ./$(BIN)
17
18 clean:                   # o. Dateien entfernen
19  rm -f $(OBJS) $(BIN)
20
21 .PHONY: clean all        # Markiert clean & all als PHONY
```

15 Anhang

15.1 Styleguide

- **Variablen, Konstanten**
 - mit Kleinbuchstaben beginnen
 - erster Buchstaben von zusammengesetzten Wörtern ist gross (mixed case)
 - keine UnderscoresBeispiele : counter, maxSpeed
- **Funktionen**
 - mit Kleinbuchstaben beginnen
 - erster Buchstaben von zusammengesetzten Wörtern ist gross (mixed case)
 - Namen beschreiben Tätigkeiten
 - keine UnderscoresBeispiele : getCount(), init(), setMaxSpeed()
- **Klassen, Strukturen, Enums**
 - mit Grossbuchstaben beginnen
 - erster Buchstaben von zusammengesetzten Wörtern ist gross (mixed case)
 - keine Underscores
 - Namen beschreiben DingeBeispiele : MotorController, Queue, Color

15.2 Code beispiele

15.2.1 String formatting

Verwendet die Output-Streams, um jene Strings im Memory zu formen. Dies ist sehr sicher, weil es das Memory völlig dynamisch handelt.

```
1 #include <sstream>
2 #include <iomanip>
3
4 std::string formatTime(int h, int m) {
5     std::ostringstream buf;
6     buf << std::setfill('0') << std::setw(2) << h
7     << ":" << std::setw(2) << m;
8     return buf.str();
9 }
```

Präsentiert modernen Standard. Kombiniert die prägnante Syntax von printf mit Streams-Sicherheit.

```
1 #include <format>
2
3 std::string formatTime(int h, int m) {
4     // :02 significa 2 cifre con riempimento zero
5     return std::format("{:02}:{:02}", h, m);
6 }
```

15.2.2 Ctor un dtor

```
1 class TString {
2 public:
3     TString();                // 1: Default Ctor
4     TString(const TString& s); // 2: Copy Ctor
5     TString(const char* p);   // 3: Ctor da stringa
6     explicit TString(int i);  // 4: Ctor esplicito da
7     int                      // ...
8 };
9
10 TString s1 = "Hoi"; // 3
```

```
11 TString s2;                // 1
12 TString s21 = s1;          // 2
13 s2 = s1;                   // Nessun Ctor (Assignment operator)
14 TString& s3 = s21;         // Nessun Ctor (Riferimento)
15 TString s4[2];             // 1, 1 (Array di 2 elementi)
16 TString* p1 = &s21;        // Nessun Ctor (Puntatore)
17 TString* p2 = new TString{"Hollo"}; // 3 (Allocazione su heap)
18 TString* p3 = new TString[3]; // 1, 1, 1 (Array di 3 su
19     heap)
20
21 TString s5[3] = {"Hoi", TString{}, s2}; // 3, 1, 2
22
23 // Esempio per il Ctor 4 (explicit)
24
25 TString s13 = TString{13}; // 4
```

15.3 Rückgabe per Referenz und Method Chaining

- **Mechanismus:** Eine Funktion sendet exakt hier eine Referenz auf das Objekt selbst per return *this; zurück. Der Return-Typ muss zwingend Referenz sein (z.B. Complex&).
- **Kaskadierung (Cascading):** Macht es möglich, sehr viele Methoden am Stück auf der gleichen Instanz pro Zeile zu rufen: obj.funz1().funz2();. **Mit einem Return by-value würde jene funz2 auf tmp Objekt angewendet**
- **Effizienz:** Im Gegensatz zu dem Return by Value entsteht gar keine Kopie der Daten (absolut kein tmp Hilfsobjekt), was massiv Ressourcen schont.
- **Persistenz:** Alle Änderungen tief in dieser Kette wirken nun wirklich ganz direkt auf das Original-Objekt statt auf eine zur Löschung verurteilte Kopie.

```
1 // .h
2 class Complex
3 {
4     public:
5     Complex(const Complex& c); // Copy constructor
6     Complex& add(const Complex& c); // adds c to the current
7     object
8 };
9 // complex.cpp
10 Complex& Complex::add(const Complex& c) {
11     this->re += c.re;
12     this->im += c.im;
13     return *this;
14 }
15
16 // main.cpp
17 Complex c3 = c1.add(c0).add(c2);
18 // c1 viene aggiornato correttamente
19 // c3 viene creato tramite copy ctor
```

15.4 Vergleich von Gleitkommazahlen (double)

Beim Vergleich von double-Werten sollte aufgrund von Rundungsfehlern **niemals der Operator == direkt verwendet werden**. Stattdessen wird geprüft, ob die absolute Differenz zwischen zwei Werten kleiner als eine definierte Toleranz (Epsilon) ist.

```
1 #include <cmath>
2
3 bool areEqual(double a, double b) {
4     static constexpr double epsilon = 1e-9;
5     return fabs(a - b) < epsilon;}
```

15.5 Non-Type Template Parameter (Static Allocation)

Diese Templates akzeptieren konstante Werte (Bsp. `std::size_t N`), die erst via **Compile-Time** evaluiert werden. Dies ist der einzige Weg, die Array-Grösse statisch festzulegen (`T data[N]`), um die hohe Flexibilität des Code zu wahren. In C++ muss der Compiler den exakten Platzbedarf vom Objekt drüben im **Stack** kennen, um Maschinen-Instruktionen mit recht fixen *Offsets* zu gen. Wäre das N Variablen-basiert, so wüsste der Compiler echt überhaupt nicht, wie viel Platz zu reservieren ist, und die Allokation fiele durch.

15.5.1 Statische Allokation vs Dynamik

Ohne Auflösung bei der Compile-Time kann der Compiler keinen Platz exakt drüben im **Stack** blockieren, was zur Nutzung vom **Heap**-Memory zwingt (das ist träger).

Eigenschaft	Via Template (statisch)	Via Konstruktor (dynamisch)
Größenlimit	Konstante am Compiletime	Variabel am Run-Time
Memory	Stack (schnell, fix)	Heap (flexibel und träge)
Verwaltung	Automatisch (0 Memory Leaks)	Manuell (new/delete)
Typ-Effekt	<code>Class<10> ≠ Class<20></code>	Der Typ bleibt identisch

15.5.2 Beispiel und Default

Möglich, einen Default-Wert zu setzen, um diesen Parameter nun absolut optional zu belassen.

```
1 template<typename T, std::size_t N = 100>
2 class Container {
3     T data[N]; // Legale solo perche N e noto al compilatore
4 };
5
6 Container<int, 20> c1; // Alloca spazio per 20 int sullo Stack
7 Container<int> c2;    // Usa il default di 100
```

15.6 Inline Functions und Templates

- **Implizites Inlining:** Alle Funktionen (ob Template o. nicht), die komplett innerhalb der Deklaration in einer Klasse (*in-class*) stehen, gelten als völlig automatisch inline durch den C++ Compiler.
- **Explizites Inlining:** Template-Fkt., die exakt ausserhalb der Klasse (*out-of-class*) stehen (auch wenn im selben Header platziert), sind absolut **nicht** ganz per Default einfach so inline. Um dies wirklich zu erzwingen, muss das `inline`-Keyword zwischen dem `template`-Tag und dem eigentlichen Return-Typ ganz fest drin stehen.
- **Vorteile und Limits:** Das Inlining tilgt exakt den Overhead des Fkt.-Aufrufs (*Zero Overhead*), ist jedoch sehr stark abgeraten für komplex-rekursive Funktionen.